Prof. Vwani Roychowdhury
UCLA, Department of ECE

Project 3: Automating Reasoning and Data Mining with Small Language Models

Due March 13, 2026 by 11:59 pm

Project Overview

This project is organized into three complementary parts that together illustrate how small language models can be adapted, integrated, and deployed for real-world data mining tasks.

- **Part A:** Improve reasoning performance of language models through parameter-efficient fine-tuning and prompting strategies.
- **Part B:** Build a structured ReAct-style agent that performs data analysis over real CSV tables with bounded context and reliable outputs.
- **Part C:** Perform regression analysis and feature engineering, integrating classical statistical modeling with agent-assisted automation.

Part A: Teaching a Small Model to Reason: LoRA Fine-Tuning, Knowledge

Introduction

State-of-the-art language models capable of complex reasoning (e.g., GPT-5, Claude, DeepSeek-R1) contain hundreds of billions of parameters, require expensive GPU clusters to serve, and are often accessible only through paid APIs. Yet many practical settings (on-device inference, cost-sensitive deployment at scale, classroom use, and privacy-constrained environments) demand models that are *small*, *fast*, and *cheap* to run. The catch is that small models, while efficient, are typically much worse at multi-step reasoning: they may understand the problem statement but fail at planning, intermediate computation, or faithful execution.

This project asks a concrete question: *Can we teach a small model to reason more like a large one, without training from scratch?* Training a new model end-to-end is unrealistic in an academic setting because it requires massive compute (thousands of GPU-hours) and enormous pre-training corpora (trillions of tokens). Instead, we start with a strong **pre-trained** or **instruction-tuned** model that already understands language and then **adapt** it to a target behavior using **Supervised Fine-Tuning (SFT)**. In SFT, the model is shown examples of the desired behavior (here: clear, step-by-step math solutions) and is trained to reproduce that style reliably.

A natural concern is cost. Even if we avoid pre-training, fine-tuning a 1.5B-parameter model can still be expensive if we update *all* weights. Full fine-tuning also risks *catastrophic forgetting*, where the model loses general capabilities while specializing on a narrow task. This motivates **parameter-efficient** methods that (i) train only a tiny fraction of parameters and (ii) leave the original model weights untouched, while still allowing the model to learn a new skill.

Parameter-Efficient Fine-Tuning: Adapting Without Rewriting the Model

Parameter-efficient fine-tuning (PEFT) methods modify how we represent an "update" to the model. Instead of changing the full weight matrices in each transformer layer, we introduce a small set of additional trainable parameters that *steer* the frozen model. This has three practical benefits:

- **Compute efficiency:** fewer trainable parameters means faster training and lower GPU memory use.
- **Storage and sharing:** only small adapter files need to be saved and distributed.
- **Stability:** freezing the base model reduces catastrophic forgetting and preserves general language competence.

In this project, we use one of the most widely adopted PEFT techniques: **Low-Rank Adaptation (LoRA)** [2].

Low-Rank Adaptation (LoRA)

LoRA is based on an empirical observation: the weight updates that matter for fine-tuning are often approximately *low-rank*. Rather than learning a full update $\Delta W \in \mathbb{R}^{d\times d}$ to a weight matrix W, LoRA represents the update as a product of two thin matrices:

$$h = (W + \Delta W)\, x = Wx + BAx, \qquad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times d},\ r \ll d.$$

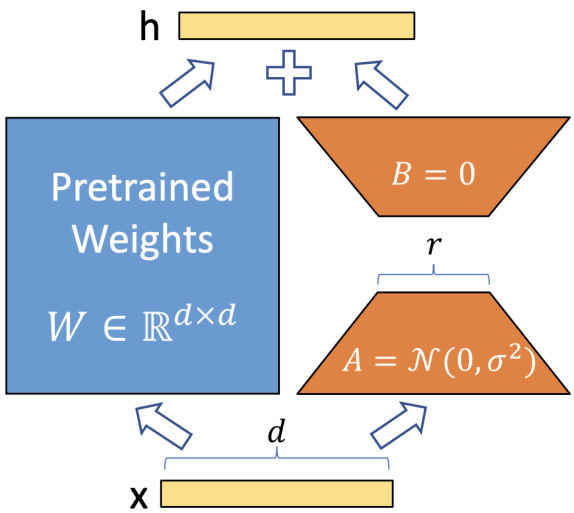

Figure 1: LoRA representation: we need to train only A and B.

Here, W is the original (frozen) weight matrix, and only the LoRA parameters A and B are trained. This changes the number of trainable parameters per adapted layer from d^2 to $2dr$, which can be orders of magnitude smaller when $r \ll d$. Concretely, for a model like Qwen2.5-1.5B with rank $r = 8$ applied to the attention layers (`q_proj`, `k_proj`, `v_proj`, `o_proj`), this corresponds to training roughly **0.14% of the total parameters**—about a ∼710× reduction in trainable parameters compared to full fine-tuning.

Practically, LoRA gives us a clean experimental knob: we can increase adapter capacity by increasing the rank r, or trade off compute versus performance by selecting which modules to adapt. Throughout this project, we use the Hugging Face **PEFT** library [6], which provides a simple interface for attaching LoRA adapters to transformer models and training only those adapter parameters.

Why Math Reasoning?

Mathematical reasoning is an ideal testbed for studying model behavior because:

- **Answers are unambiguous.** Unlike open-ended generation, a math answer is either correct or incorrect, so evaluation is straightforward.
- **Multi-step reasoning is required.** Even grade-school word problems demand decomposition, intermediate computations, and a final synthesis step.
- **Failures are diagnosable.** When a model gets the wrong answer, we can often localize the error (e.g., arithmetic slip, flawed logic, or misunderstanding the question).

The GSM8K Dataset

We use **GSM8K** (Grade School Math 8K) [1], a widely-used benchmark of grade-school level math word problems. The dataset contains **7,473 training examples** and **1,319 test examples**. Each problem requires 2–8 steps of elementary arithmetic and reasoning, and the answer is always an integer. Here is a representative example:

GSM8K Example

Question: Betty is saving money for a new wallet which costs \$100. Betty has only half of the money she needs. Her parents decided to give her \$15 for that purpose, and her grandparents twice as much as her parents. How much more money does Betty need to buy the wallet?

Answer: In the beginning, Betty has only $100/2 = \$50$. Betty's grandparents gave her $15 \times 2 = \$30$. This means Betty needs $100 - 50 - 30 - 15 = \$5$ more. $\boxed{5}$

Our Model: Qwen2.5-1.5B-Instruct

Throughout this project, we use **Qwen2.5-1.5B-Instruct** [3], a small instruction-tuned language model from Alibaba with 1.5 billion parameters. It has been pre-trained on a large text corpus and instruction-tuned for general chat—but it has *not* been specifically trained for mathematical reasoning. This makes it an ideal "student" model: competent at language but weak at math.

The Gap We Want to Close

To see the problem concretely, consider the following question from the GSM8K test set, and how our base model handles it compared to a strong model (GPT-4.1-mini):

Base Model (Qwen2.5-1.5B-Instruct) — INCORRECT

Question: Every day, Wendi feeds each of her chickens three cups of mixed chicken feed. She gives the chickens their feed in three separate meals. In the morning, she gives her flock of chickens 15 cups of feed. In the afternoon, she gives her chickens another 25 cups of feed. How many cups of feed does she need to give her chickens in the final meal of the day if the size of Wendi's flock is 20 chickens?

Model's reasoning (excerpt):
Total feed = Morning feed + Afternoon feed
$15 + 25 = \mathbf{7}$ cups $\longleftarrow$ Elementary arithmetic error!
...
Final meal $= 60 - 7 = 53$

Model's answer: $\boxed{53}$ (Ground truth: $\boxed{20}$)

The model sets up the problem correctly (each chicken gets 3 cups, 20 chickens $\rightarrow$ 60 cups total) but then makes a catastrophic arithmetic error: $15 + 25 = 7$. This cascades through the rest of the solution.

Now consider GPT-4.1-mini solving a similar problem:

GPT-4.1-mini —

Question: Betty is saving money for a new wallet which costs \$100. Betty has only half of the money she needs. Her parents decided to give her \$15 for that purpose, and her grandparents twice as much as her parents. How much more money does Betty need to buy the wallet?

GPT's reasoning:

1. The wallet costs 100.
2. Betty has half: $\frac{100}{2} = 50$
3. Parents give her 15.
4. Grandparents give $2 \times 15 = 30$
5. Total: $50 + 15 + 30 = 95$
6. Still needs: $100 - 95 = 5$

Answer: $\boxed{5}$ ✓

Notice the difference: GPT's solution is *clearly structured*, uses *numbered steps*, performs *correct arithmetic*, and presents the answer in a clean format. **The goal of this project is to teach our small model to reason like this**—through fine-tuning, prompting strategies, and knowledge distillation.

Project Roadmap

- **Task 1:** Benchmark the base model to establish the performance floor.
- **Task 2:** Apply LoRA fine-tuning with GSM8K training data and study scaling behavior.
- **Task 3:** Investigate few-shot prompting and its interaction with fine-tuning.

- **Task 4:** Discover that *data quality* matters more than *data quantity* through knowledge distillation, and attempt to push accuracy further.

Setup and Evaluation Protocol

All experiments in this project follow a common evaluation protocol to ensure results are comparable across models and training setups.

System Prompt Guidelines

Because GSM8K problems require multi-step reasoning, we strongly recommend using a **system prompt** during evaluation (and matching it during training). In practice, a well-designed system prompt often improves reliability by enforcing a consistent reasoning style and a predictable final-answer format that can be parsed automatically.

Your system prompt should do two things:

1. **Elicit structured reasoning:** promote step-by-step work and intermediate calculations.
2. **Encourage a consistent answer template:** ask for a clearly marked final answer (using a stable marker) so automated extraction is reliable.

In practice, adding lightweight formatting constraints often improves both (i) reasoning clarity and (ii) evaluation robustness. Common, easy-to-parse conventions include:

- Ending with a clearly labeled final answer such as `Answer: <answer>` or `Final: <answer>`.
- Using a single LaTeX marker like `\boxed{...}` for the final answer.

We encourage you to experiment with 2–3 prompt variants early (e.g., different answer markers or step formatting), then **choose one prompt format and keep it fixed** for the remainder of the project. Keeping the prompt fixed ensures results remain directly comparable across models and training regimes.

Evaluation Protocol

- **Test set:** 100 questions from the GSM8K test split (fixed across all experiments for compute efficiency).
- **Metric:** Accuracy = (number of correct answers) / (total questions).

Compute Environment

Note: It is highly recommended you apply for Colab Pro with your student IDs right away, as this project requires heavy GPU usage!

All experiments can be run on Google Colab. Approximate runtimes on a T4 GPU:

- Evaluation (100 questions): $\sim$20 minutes

- LoRA training (1,000 examples, 1 epoch): ∼40–50 minutes
- LoRA training (3,000 examples, 1 epoch): ∼80–90 minutes
- LoRA training (7,473 examples, 1 epoch): ∼3–4 hours

Task 1 — Baseline: How Good is the Base Model?

Before any fine-tuning, we first measure how well the base Qwen2.5-1.5B-Instruct model solves GSM8K problems *out of the box*. This establishes a **performance floor** that all subsequent interventions (fine-tuning, prompting, distillation) will be compared against.

Task 1.1 Baseline Evaluation

Evaluate the base model on a fixed subset of 100 GSM8K test questions using the project's standardized prompting and answer-extraction setup.

Note: To reduce inference time, use batching. A batch size of 16 is recommended (and works well on Colab). With batching, 100 questions typically take about 25 minutes. As always, test performance and confirm your batching syntax on a small sample before running the full code.

QUESTION 1: (10 points) Run the base Qwen2.5-1.5B-Instruct model on 100 GSM8K test questions and report the accuracy. You should expect approximately 35–40%. *(Exact values may vary slightly depending on the prompt format and extraction rule.)*

QUESTION 2: (10 points) Inspect at least 3 cases where the base model produces an **incorrect** answer. For each example, include:

- The question,
- A short excerpt of the model's solution highlighting the failure,
- The extracted answer vs. the ground-truth answer.

Classify each failure mode (e.g., arithmetic slip, reasoning/logical error, misunderstanding the problem, formatting/extraction issue). Do you observe any recurring patterns?

Task 2 — LoRA Fine-Tuning on GSM8K

The base model struggles with math. Can we improve it by showing it thousands of worked examples? In this task, you will apply LoRA SFT using the GSM8K training split, starting small and scaling up to study how data quantity affects performance.

Task 2.1 Understanding the Training Pipeline

Before training anything, take time to inspect the helper code and understand *what is actually being optimized*. LoRA SFT is conceptually simple, but many failures in practice come from not understanding the data formatting, where adapters are applied, and which tokens contribute to the loss.

Conceptually, the training pipeline has three key ingredients:

1. **Data formatting:** Each GSM8K training example is converted into a chat-style interaction (system prompt + user question + assistant solution). The assistant solution should follow a consistent answer convention so evaluation can extract answers reliably.
2. **LoRA configuration:** LoRA adapters are inserted into selected linear layers (commonly attention projections and MLP projections). The base weights remain frozen, and only the adapter parameters are trained.
3. **Completion-only loss:** The loss is computed only on the assistant's response tokens (not on the system or user tokens). This focuses training on producing better solutions rather than memorizing the prompt.

Default hyperparameters are summarized below. Use these unless a question explicitly asks you to experiment.

Hyperparameter	**Default**	**Notes**
LoRA rank (r)	8	Controls adapter capacity
LoRA alpha	16	Scaling factor (rule of thumb: $2r$)
LoRA dropout	0.05	
Learning rate	2×10^{-4}	Cosine schedule with 5% warmup
Epochs	1	
Per-device batch size	8	
Gradient accumulation	4	Effective batch size = 32
Max sequence length	1024	
Target modules	Attention only	`q,k,v,o_proj`

QUESTION 3: (15 points) Pick the three hyperparameters (LoRA rank, LoRA alpha, Gradient accumulation) from the table above and explain:

- what each hyperparameter controls,
- what you expect to happen if you **increase** it,
- what you expect to happen if you **decrease** it.

Your answer should reflect practical tradeoffs (e.g., compute/memory, stability, overfitting vs. underfitting).

QUESTION 4: (15 points) Report:

(a) the total number of parameters in the base model,

(b) the number of trainable LoRA parameters under the default configuration,

(c) the percentage of parameters being trained.

Briefly explain why this percentage is small and how LoRA achieves this reduction.

Task 2.2 Training with Increasing Data

The GSM8K training split contains 7,473 question-answer pairs. Rather than starting at full scale, begin with a smaller run to build intuition, validate your setup, and measure the first performance jump.

QUESTION 5: (25 points) Train a LoRA SFT model using **1,000** training examples. Evaluate on 100 GSM8K test questions and report the accuracy. Include a brief comment on whether the improvement over the baseline matches your expectations.

Scaling up training data improves performance, but it also increases runtime and cost. Before running larger experiments, it is worth forming a hypothesis about whether scaling will meaningfully help.

QUESTION 6: (10 points) Hypothesis question (write before running larger training): Do you think scaling from 1,000 examples to **3,000** and/or **all 7,473** examples is worth the additional compute? What do you expect the accuracy gains to look like (roughly), and why? In your answer, discuss whether you would scale in multiple steps or jump directly to the full dataset.

QUESTION 7: (20 points: 5 + 5 + 10) Now scale up your training data (recommended: 3,000 examples, and optionally the full 7,473). Evaluate each trained model on the same 100-question test subset and report the accuracies.

Finally, plot accuracy as a function of the number of training examples (x-axis: 0, 1000, 3000; y-axis: accuracy). Describe the trend you observe and comment on diminishing returns in data scaling for SFT.

Task 2.3 Qualitative Analysis

QUESTION 8: (10 points) Compare the base model and your best SFT model on the same 3 failure examples you identified in Task 1. For each example, show both models' responses side by side. Does the SFT model fix any of these errors?

QUESTION 9: (5 points) Identify 2 examples where your best SFT model **still fails**. What types of errors persist after fine-tuning? Does the model struggle more with arithmetic, multi-step reasoning, or problem comprehension?

Note: You should expect to reach around 45-50% accuracy by this time.

Task 3 — Few-Shot Prompting

Fine-tuning changes the model's weights. An orthogonal approach is to change the *prompt*: by including a small number of worked examples ("shots") before the actual question, we can steer the model toward a desired reasoning style and a more consistent output format at inference time, without updating any parameters.

Task 3.1 Few-Shot Setup

In k-shot prompting, we prepend k complete (question, solution) pairs as user/assistant turns in the chat before the actual test question. Conceptually, the model sees:

```
[system]    System prompt
[user]      Example question 1
[assistant] Example solution 1
```

```
[user]      Example question 2
[assistant] Example solution 2
...
[user]      Actual test question
[assistant] <model generates here>
```

For fairness, use a **fixed pool of k demonstration examples** for all models you evaluate (base, SFT). The goal is to keep the prompt content constant so that differences in performance reflect differences in the model rather than differences in the examples.

Practical note: The quality of your demonstrations matters. A natural question to ask is: *what is the best source for high-quality step-by-step reasoning answers to use as demonstrations?* Yes, that is correct — it *is* the place to go now. Any guesses?

Task 3.2 Few-Shot Experiments

QUESTION 10: (20 points) Evaluate k-shot prompting (use $k = 3$) on:

1. the base model,
2. your LoRA SFT model trained on 3k examples,

Report the k-shot results alongside the corresponding no-demonstration baseline results, and compute the improvement (Δ).

QUESTION 11: (15 points) Analyze the effect of few-shot prompting on each model:

- Does few-shot help the **base model**? If not, why might it perform *worse* with demonstrations?
- Does few-shot help the **SFT models**? By how much?
- Which model benefits the most from few-shot prompting, and why?

Task 4 — Beyond Scaling: Quality Matters (Open-Ended Exploration)

By now, you should have improved substantially over the base model using LoRA SFT and few-shot prompting. However, you may also have noticed a recurring pattern: **scaling** (more training examples, longer prompts, more shots) helps, but the gains can slow down. At some point, the bottleneck is less about *how much* data you have and more about *how good* the supervision is.

This sets up the central idea for the final part of the project: **improving reasoning performance is often a data-quality problem**. If your training solutions are noisy, terse, poorly structured, or inconsistent, then adding more of them may not teach the model the behavior you want. In contrast, training on fewer but higher-quality, well-structured solutions can lead to surprisingly large improvements.

Using a quality-focused approach that is feasible within the scope of this course, we were able to obtain a model in the **low-to-mid 70%** accuracy range on this evaluation setup. This number serves as a practical reference point.

QUESTION 12: (10 points) **Qualitative reflection (short):** Based on your results so far, what do you think is limiting performance? For each of the following, justify with 2–4 concrete observations from your own failures:

- arithmetic reliability,
- multi-step planning / long-horizon reasoning,
- problem comprehension,
- output consistency / extraction failures,
- training data quality (solution style, structure, or noise).

Task 5 — Open Challenge: Push Toward the Ceiling

Your goal is to **beat your own best score from Tasks 1–3**. This final task is intentionally exploratory: you will propose a hypothesis, run a small set of targeted experiments, and report what worked (or did not work) and why.

Grading emphasis: This task rewards **scientific reasoning** more than brute-force compute.

- **25 points:** quality of hypotheses, experimental design, and analysis (clear ablations, controlled variables, honest interpretation).
- **10 points:** final accuracy improvement (on the same 100-question evaluation subset).
- **Bonus (+10):** if you surpass the staff reference number, you receive a bonus on top of the project score.

Below are **sensible directions** to explore (choose one or combine a small number of them):

- **Demonstration quality:** curate a small set of high-quality few-shot demonstrations (clear steps, correct arithmetic, consistent final-answer marker) and measure gains over your current few-shot setup.
- **Prompt and format engineering:** redesign the system prompt and/or answer marker to reduce extraction failures and encourage disciplined reasoning; test a small set of prompt variants and lock one in.
- **Targeted training data selection:** train on a subset of GSM8K that emphasizes harder multi-step problems or known failure modes of the base model.
- **Self-consistency at inference:** sample multiple solutions (temperature > 0) and take a majority vote over extracted answers.
- **Light hyperparameter tuning:** adjust LoRA rank, learning rate, epochs, or target modules; justify expected effects and report what changed.
- **Quality-focused supervision:** generate or obtain a set of higher-quality step-by-step solutions and fine-tune on them (filtering for correctness when possible).

QUESTION 13: Design and implement **your favourite strategy** to improve upon your best score. In your report, include:

(a) **Hypothesis:** what you think will help and why.

(b) **Method:** what you changed (prompting, training data, hyperparameters, inference procedure, etc.).

(c) **Results:** accuracy on the same 100-question evaluation subset (include your baseline for comparison).

(d) **Analysis:** what you learned from the experiment(s), including at least one failure mode or unexpected result.

==========

Part B: Agentic Data Mining with ReAct

Introduction

Modern data mining pipelines increasingly operate at a scale where **manual curation is no longer feasible**. In many real-world settings—customer logs, scientific datasets, public records, transaction streams, and web data—the raw material is noisy, heterogeneous, and only *partially* structured. Traditionally, teams would write domain-specific feature extraction and analytics code by hand, but today we increasingly rely on **AI agents** to help *discover*, *clean*, *transform*, and *analyze* data automatically.

This sub-project explores a practical question in modern large-scale data mining:

> *Can we build an LLM-based agent that answers data analysis questions by interacting with real CSV tables, while producing outputs that are reliable enough to be evaluated automatically?*

The project is motivated by a practical tension:

- **LLMs are flexible**, but their outputs can be unpredictable and hard to parse.
- **Data analysis requires precision**: a single wrong filter, join, or aggregation can flip an answer.
- **Large-scale settings demand automation**: if we cannot automate the whole workflow, the system does not scale.

What You Will Build

You will build a small **data analysis agentic system** that solves questions about CSV files. The system follows the **ReAct** paradigm (Reasoning + Acting): it iteratively (i) decides what to do next, (ii) writes Python code to perform that step, (iii) executes the code, (iv) summarizes the tool output into a structured observation, and (v) uses that observation to continue or stop with a final answer.

You will also incorporate **structured output** (via **Outlines**), enabling robust orchestration and automatic evaluation.

Learning Objectives

By the end of this project, you should be able to:

- Work with a real benchmark for evaluating LLM agents on data analysis tasks.
- Implement a basic agentic system with planning, tool-use (code), and observations.
- Use structured generation to guarantee parseable intermediate outputs.
- Evaluate an agent quantitatively and analyze typical failure modes (format errors, reasoning errors, code bugs, data issues).

Dataset: InfiAgent-DABench (DAEval)

We use the **InfiAgent-DABench** benchmark. Each example consists of:

- A natural-language **question** about a CSV table;
- Additional **constraints** (e.g., rounding rules, filtering rules);
- A required **output format** (so answers can be checked automatically);
- A reference CSV file (one of many, with different schemas);
- Ground-truth answers in a closed-form representation.

The dataset is provided in this link to you in the following directory layout:

- `da-dev-questions.jsonl` *(questions + metadata)*
- `da-dev-labels.jsonl` *(ground-truth answers)*
- `da-dev-tables/` *(CSV files referenced by questions)*

Important: The CSV files do *not* share a common schema. One question may reference a finance table with columns like `revenue` and `quarter`, while another may reference a sports play-by-play table with completely different fields. This is a realistic large-scale data mining challenge: **heterogeneous schemas are the norm, not the exception.**

Why Structured Output? (Outlines)

A typical pipeline has multiple components: a planner that decides the next step, a coder that writes code, and an executor that runs code. If the planner emits free-form text like:

```
I think we should compute the mean fare and round to two decimals.
```

then your program must *guess* what to do next. This is not-ideal: the agent might change phrasing across runs, break your parser, or produce ambiguous instructions.

Instead, we will use **structured output** so the planner emits a machine-readable object, for example:

```
{"thought": "...", "is_done": false, "response": "Compute the mean of
Fare."}
```

In this project, you will use **Outlines** to constrain an open-source LLM to generate outputs that follow a **Pydantic schema**. This eliminates a major source of agent instability: parsing errors.

Agents and Agentic Data Mining

In this project, an **agent** is a system that:

1. maintains state (what it has done so far),
2. decides what to do next,
3. interacts with external tools (here: Python execution over CSV files),
4. uses observations from tools to update its next decision,
5. terminates when it can produce a final answer.

Compared to a single one-shot LLM response, an agent can:

- **maintain and manage context**: keep a persistent working memory (tool outputs, intermediate results, constraints), selectively summarize it, and feed back only the most relevant state **when the context window is limited (this is especially important when we are using smaller models)**;
- **use specialized prompting per role**: separate prompts for planning, coding, and verification, so each step is more focused than a single all-in-one prompt;
- **decompose** complex tasks into smaller steps,
- **ground** answers in actual computations (reducing hallucinations),
- **debug** when code fails (recover from errors and retry),
- provide **transparent traces** (what it attempted and what it observed).

The ReAct Idea (Reasoning + Acting)

ReAct interleaves two things:

- **Reasoning:** decide what to do next given the current state and past observations.
- **Acting:** invoke an external tool (here: execute Python code over CSV files).

In our setting, the **action** is Python code that reads a CSV and computes statistics. After execution, instead of directly feeding the raw stdout/stderr back to the planner, we introduce an additional **Observation agent** that converts tool output into a concise, task-relevant *summary of observation* (e.g., key numbers, detected schema issues, error type, and suggested fix) (We bring in observation agent to avoid context window become too big for certain complex tasks). The planner then uses this structured observation to decide the next step or terminate with the final formatted answer (e.g., `@mean_fare[34.65]`).

Setup and Starter Code

You may work in Google Colab or locally. Your solution must be runnable from a clean environment. We recommend:

- Python $\geq$ 3.10
- `transformers`, `accelerate`, `torch`
- `pandas`, `numpy`
- `pydantic`
- `outlines`

Compute note: A 4B-parameter open model is typically feasible on a single GPU (e.g., Colab T4/L4/A100), but generation can still be slow.

Safety note: Executing LLM-generated code can be dangerous. In this project:

- Do **not** allow internet access from the execution environment.
- Restrict code to reading the provided CSV files only.
- Avoid writing files, deleting files, or calling system commands.

Model Requirement

Use `Qwen/Qwen3-4B-Instruct-2507` (or any model under 5B parameters). You should **load the model once** and reuse it for all roles in your pipeline. Different agents (Planner/-Coder/Observer) are implemented by **different prompts** and may use **different generation paradigms** (e.g., structured outputs via Outlines + Pydantic for Planner and Observer, code-focused prompting for Coder).

Task 1 — Dataset Inspection and Sanity Checks

Before building an agent, let's understand the dataset format.

Task 1.1 Load and Inspect JSONL Files

QUESTION 14: Load `da-dev-questions.jsonl` and `da-dev-labels.jsonl`. Report:

- The number of questions and labels.
- The set of keys present in a question record (print one example).
- The set of keys present in a label record (print one example).

Task 1.2 Inspect the Table References

QUESTION 15: Pick 3 random question IDs. For each:

- Print the `file_name` of the referenced CSV.
- Load the CSV with pandas and print `df.shape`, `df.dtypes`, and `df.head(3)`.
- Print the corresponding question.

Task 1.3 Understand the Required Answer Format

QUESTION 16: Find 2 examples where the required format contains multiple answer slots (e.g., two or more `@name[value]` fields). Explain:

- How the dataset represents multi-part answers in the labels.
- How you plan to evaluate such answers automatically.

Task 1.4 Checking the subset

QUESTION 17: Unfortunately, the model we are going to use is still not powerful enough to solve all the tasks. Here we are selecting 10 sub-tasks that are proved to be solveable:

The selected IDs are: `SELECTED_IDS = [0, 5, 9, 10, 14, 18, 24, 25, 26, 55]`.

- Print out and check those tasks.

Task 2 — Model Loading and Structured Output: Make the Planner Parseable

Please Refer to the helper code for reference.

First, let's load the model we are going to use: `MODEL_NAME = "Qwen/Qwen3-4B-Instruct-2507"`.

Next, build a structured **Planner** component that returns a JSON object following a Pydantic schema such as:

```
class PlannerOutput(BaseModel):
    thought: str
    is_done: bool
    response: str
```

You must use **Outlines** (or an equivalent constrained decoding approach) so that **the JSON is always valid**. You can refer to the helper code for how to use outlines.

QUESTION 18: Demonstrate (with 5 different prompts) that your planner always returns valid JSON that parses into your Pydantic model *without* try/except fallbacks. Include at least one case where the planner decides it is done (`is_done=true`).

QUESTION 19: Explain in a few sentences why structured output is useful for large-scale data mining pipelines.

Task 3 — Build a ReAct Data Analysis Agent

Implement a 4-part system:

- **Planner (Reasoning):** decides the next analysis step, or stops with the final formatted answer.
- **Coder (Action):** writes Python code for the planner's instruction.
- **Executor (Tool):** runs the code and returns raw stdout/stderr (or an error trace).
- **Observer (Structured Observation):** converts raw tool output into a concise, task-relevant observation that the planner can reliably use.

Your agent must run a loop for up to `max_steps=5` iterations:

1. Planner produces structured output (`thought`, `is_done`, `response`).
2. If `is_done=true`, return `response` as the final answer.
3. Otherwise, Coder writes Python code based on Planner's response and Executor runs it to obtain raw stdout/stderr.
4. Observer reads the raw output and the planner's original response and produces a observation summary (e.g., extracted values, warnings, error category, next-step hint).
5. Append the quadruple (instruction, code, raw output, observation summary) to the history and continue.

Required Features

Required Features

- **Error recovery:** if the executor returns an error (e.g., parsing error, missing column, type conversion, runtime exception), the system must use the error message as feedback and attempt to fix the code and retry, instead of immediately failing.
- **Bounded context (runnable on Google Colab):** the system must avoid unbounded growth of the prompt/history. In particular, it should remain runnable on a single Tesla T4 GPU with 15 GB GPU memory. Please optimize your design of the agents, especially the observation summarization agent.
- **Structured outputs for planner:** planner must return a machine-parseable object with a fixed schema (recommended: Pydantic + Outlines).

QUESTION 20: Run your ReAct agent on the 10 tasks. Report:

- Accuracy
- At least 3 qualitative traces (planner thought, code, observation, final answer) that illustrate interesting behaviors: success, failure, recovery from an error.

Part C: Regression Analysis

1 Introduction

Regression analysis is a statistical procedure for estimating the relationship between a target variable and a set of features that jointly inform about the target. In this sub-project, we explore specific-to-regression feature engineering methods and then **reuse your ReAct agent** to train regression models.

2 Datasets

Valentine's day might be over, but we are still interested in building a bot to predict the price and characteristics of diamonds. A synthetic diamonds dataset can be downloaded from this **link**. This dataset contains information about 150000 round-cut diamonds. There are 14 variables (*features*) and for each sample, these features specify the various properties of the sample. Below we describe some of these features:

- **carat**: weight of the diamond
- **cut**: quality of the cut
- **clarity**: measured diamond clarity
- **length**: measured length in mm
- **width**: measured width in mm
- **depth**: measured depth in mm
- **depth_percent**: diamond's total height divided by it's total width
- **table_percent**: width of top of diamond relative to widest point
- **gridle_min**: refers to the thinnest part of the girdle
- **gridle_max**: refers to the thickest part of the girdle

In addition to these features, there is the target variable: i.e what we would like to predict:

- **price**: price in US dollars

3 Training Pipelines

In this section, we describe the setup you need to follow. Follow these steps to process the datasets.

3.1 Data Inspection

Before training an algorithm, it's always essential to inspect the data. This provides intuition about the quality and quantity of the data and suggests ideas to extract features for downstream ML applications.

QUESTION 21: Perform an exploratory data analysis on the provided Diamonds dataset. Report the following:

- **Correlation Analysis:** Plot a heatmap of the Pearson correlation matrix. Report which features have the highest absolute correlation with the target variable (`price`). Briefly describe what the correlation patterns suggest.
- **Distribution Analysis:** Plot the histogram of numerical features. Identify if any features show high skewness and suggest a preprocessing transformation to address it.
- **Categorical Analysis:** Construct box plots of categorical features versus the target variable. Describe any significant trends (e.g., how `cut` or `color` affects the price range).

3.2 Handling Categorical Features

A categorical feature is a feature that can take on one of a limited number of possible values. If one dataset contains categorical features, a preprocessing step needs to be carried to convert categorical variables into numbers and thus prepared for training.

One method for numerical encoding of categorical features is to assign a scalar. For instance, if we have a "Quality" feature with values {`Poor, Fair, Typical, Good, Excellent`} we might replace them with numbers 1 through 5. If there is no numerical meaning behind categorical features (e.g. {`Cat, Dog`}) one has to perform "one-hot encoding" instead.

QUESTION 22: Explain the following trade-off questions.

- Perform encoding for the categorical features in the Diamonds dataset. Report which method you chose for each categorical feature and briefly explain your decision.
- Explain the following trade-offs:
  - What information does one-hot encoding discard?
  - What assumption should hold strongly if we perform the scalar encoding instead?

3.2.1 Standardization

Standardization of datasets is a common requirement for many machine learning estimators; they might behave badly if the individual features do not more-or-less look like standard normally distributed data: Gaussian with zero mean and unit variance. If a feature has a variance that is orders of magnitude larger than others, it might dominate the objective function and make the estimator unable to learn from other features correctly as expected.

QUESTION 23: Standardize feature columns and prepare them for training. Save your standardized version of the dataset as `diamonds_standardized.csv`.

3.2.2 Feature Selection

- `sklearn.feature_selection.mutual_info_regression` function returns estimated mutual information between each feature and the label. Mutual information (MI) between two random variables is a non-negative value which measures the dependency between the variables. It is equal to zero if and only if two random variables are independent, and higher values mean higher dependency.
- `sklearn.feature_selection.f_regression` function provides F scores, which is a way of comparing the significance of the improvement of a model, with respect to the addition of new variables.

You **may** use these functions to select features that yield better regression results (especially in the classical models).

QUESTION 24: Print the top 5 features using each method (`mutual_info_regression` and `f_regression`).

- **Agentic Integration:** For this step, load `diamonds-questions.jsonl` and `diamonds-labels.jsonl` (question id 0 and 1) and use your ReAct agent from Part 2 to automatically identify and print the top features. If the agent gets stuck, you may manually write the code to compute and print them.

From this point on, you are free to use any combination of features, as long as the performance on the regression model is on par (or slightly worse) than the Neural Network model.

Save your selected feature new csv as `diamonds_selected.csv`.

3.3 Training

Once the data is prepared, we would like to train multiple algorithms and compare their performance using average RMSE from 10-fold cross-validation (please refer to part 3.4).

3.4 Evaluation

Perform 10-fold cross-validation and measure average RMSE errors for training and validation sets.

Task 5.1 Linear Regression

QUESTION 25: Agentic Integration: For this step, load `diamonds-questions.jsonl` and `diamonds-labels.jsonl` (question ids 2, 3, and 4) and use your ReAct agent from Part 2 to automatically train the models and extract the necessary metrics. If the agent gets stuck, you may manually write the code to complete the training.

Important: List out the Python code generated by your agent for these tasks. Review the code to ensure it makes sense and correctly implements the requested regression models. Do not blindly trust the agent's output; verify its logic before proceeding.

What is the objective function? Train three models: (a) ordinary least squares (linear regression without regularization), (b) Lasso and (c) Ridge regression, and answer the following questions.

- Explain how each regularization scheme affects the learned parameter set.
- Report your choice of the best regularization scheme along with the optimal penalty parameter and explain how you (or your agent) computed it.
- Some linear regression packages return p-values for different features[1]. What is the meaning of these p-values and how can you infer the most significant features? A qualitative reasoning is sufficient.

Submission

Please submit a PDF report to Gradescope via BruinLearn. You can find a link to Gradescope in the left panel on BruinLearn.

In addition, please submit a zip file containing your **report** and your **code** with a **readme file** explaining how to run your code to BruinLearn. The zip file should be named `Project3_UID1_UID2_...` where `UIDx`'s are the student ID numbers of the team members. Only one submission per team is required. If you have any questions, please ask on Piazza or through email.

References

[1] Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., & Schulman, J. (2021). Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*.

[2] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-Rank Adaptation of Large Language Models. *ICLR 2022*.

[3] Yang, A., Yang, B., Hui, B., Zheng, B., Yu, B., Zhou, C., ... & Lin, J. (2024). Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115*.

[4] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., & Zhou, D. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *NeurIPS 2022*.

[5] Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., ... & Amodei, D. (2020). Language Models are Few-Shot Learners. *NeurIPS 2020*.

[6] Hugging Face. PEFT: Parameter-Efficient Fine-Tuning. https://huggingface.co/docs/peft.

[7] Hu, X., Zhao, Z., Wei, S., Chai, Z., Ma, Q., Wang, G., Wang, X., Su, J., Xu, J., Zhu, M., Cheng, Y., Yuan, J., Li, J., Kuang, K., Yang, Y., Yang, H., & Wu, F. (2024). InfiAgent-DABench: Evaluating Agents on Data Analysis Tasks. *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*.

[8] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR 2023*. https://react-lm.github.io

[9] Outlines Developers. Outlines: Structured outputs for LLMs. https://dottxt-ai.github.io/outlines/

[1] E.g: `scipy.stats.linregress` and `statsmodels.regression.linear_model.OLS`

[10] Wolf, T., et al. (2020). Transformers: State-of-the-art Natural Language Processing. *EMNLP 2020: System Demonstrations*.